

Coding Guidelines

The rules of libterae's C code, and how each one is verified

Emilio Schininà, M.Sc. in Automation Engineering
Version 0.1
November 8, 2026

Copyright © 2026 Emilio Schininà, M.Sc. in Automation Engineering. libterae is licensed under the GNU General Public License, version 3 or any later version, or under a commercial license.

Contents

1 Coding Style **1**
1.1 The language . . . 2
1.2 Categories and deviations. . . . 2
1.3 Rules that most shape the code . . . 3
1.4 Copyright and licence . . . 4
1.5 Documentation comments . . . 5
1.6 Additional coding style rules . . . 8
1.6.1 The rest of the additional style rules . . . 9

2 Project Rules **11**
2.1 Where code goes. . . . 11
2.2 What the code may call . . . 13
2.3 Programs and tests. . . . 14
2.4 Protocol behaviour . . . 15
2.5 Text from elsewhere . . . 15
2.6 Working with the checks . . . 16

3 Automated Verification **17**
3.1 Kinds of automated check. . . . 17
3.2 Verifying a change . . . 18
3.3 The checks' own rules . . . 18

4 MISRA C:2012, Guideline by Guideline **21**
4.1 Reading a row . . . 21
4.2 Directives. . . . 22
4.3 The language standard . . . 23
4.4 Unused and dead code . . . 24
4.5 Comments . . . 25
4.6 Characters and escape sequences . . . 25
4.7 Identifiers . . . 26
4.8 Bit-fields . . . 26

coding-guidelines-en.tex Page iv November 8, 2026 13:15

4.9 Literals and constants . . . 27
4.10 Declarations and definitions . . . 27
4.11 Initialisation . . . 29
4.12 Essential types . . . 29
4.13 Pointer conversions . . . 30
4.14 Expressions . . . 31
4.15 Side effects . . . 32
4.16 Controlling expressions . . . 33
4.17 Control flow . . . 33
4.18 Switch statements . . . 34
4.19 Functions . . . 35
4.20 Pointers and arrays . . . 36
4.21 Overlapping storage . . . 37
4.22 The preprocessor . . . 37
4.23 The standard library . . . 38
4.24 Resources and threads . . . 41
4.25 Generic selections . . . 43

5 Coverage at a Glance 45
5.1 Guidelines by how they are verified . . . 45
5.2 Known misses . . . 46
5.3 What a person reviews . . . 46
5.4 What only a person verifies . . . 47

1

Coding Style

These guidelines are the set of rules that libterae's C code follows, from design through development to maintenance. Every rule says what it asks for and how it is verified: by an automated check, by a person reviewing the code, or by both, the automated check finding some violations and the review covering the rest. What a rule is verified with, and to what end, is never part of the rule.

This document is in five chapters:

- **Chapter 1, Coding Style.** gives the language and the standard it is written in, MISRA's categories and how a deviation from a rule is recorded, the guidelines that most shape the code, copyright and licence, documentation comments, and the additional coding style rules.
- **Chapter 2, Project Rules.** says where code goes, what it may call, programs and tests, protocol behaviour, text from elsewhere, and how a change is verified before it is called done.
- **Chapter 3, Automated Verification.** lists the kinds of automated check the project develops, what a change is verified with, and the rules the checks themselves follow.
- **Chapter 4, MISRA C:2012, Guideline by Guideline.** lists all 221 guidelines, one at a time, in the project's own words, each with its category, how it is verified, and what shows that the verification works.
- **Chapter 5, Coverage at a Glance.** counts the guidelines by how they are verified, what a person still has to review, and the violations a check is known to miss.

In the tables, the last column says how a rule is verified. *Automated* means that a check reports the violation and names the rule. *A person* means that a reviewer has to verify it. *In part* means that a check finds some violations, and a person still has to review the rest.

coding-guidelines-en.tex Page 2 November 8, 2026 13:15

1.1 The language

libterae is written in C99 and follows MISRA C:2012. The guidelines it is held to are the 2013 edition with Amendments 1 and 2: 175 guidelines. Amendments 3 and 4 add 46 more. They are followed as well, and a violation of them is reported like any other, but the project's claim is for Amendments 1 and 2, and Chapter 5 counts the later ones apart, as tracked. Of those 46, 29 are about C11 features, which C99 code can't use, so they don't arise.

The library is `src/` and `include/`: MISRA C:2012 is adopted for it, with Amendments 1 to 4. `platform/` lies outside the MISRA claim.

Table 1.1 The language

Rule	What it means	Verified by
C99	The code is C99, the dialect the project is written in, with no compiler extensions and nothing from C11.	Automated: the code is compiled with `-std=c99 -pedantic-errors`, and C11's thread and atomic headers are rejected.
MISRA	MISRA C:2012 with Amendments 1 to 4 is followed, rule by rule, as Chapter 4 lists.	Automated, for 184 of the 221 guidelines, 47 of them in part. A person, for 6. Chapter 5 counts them.
Compliance	A clean run of the checks is not a claim of compliance. The guidelines verified in part, or not at all, need a review.	A person.

1.2 Categories and deviations

Each MISRA guideline has a category, which says how strictly it is followed here:

Table 1.2 MISRA's categories in this project

Mandatory	Always followed. No deviations.
Required	Followed unless a deviation is recorded.
Advisory	Treated the same as Required.

A deviation is a recorded exception to a Required or Advisory rule, for a case where it can't reasonably be followed. MISRA publishes deviation permits that already cover

coding-guidelines-en.tex Page 3 November 8, 2026 13:15

some common cases, mostly for generated code, and an entry may cite one, but most do not fit code that people write and review.

Table 1.3 Deviations

Rule	What it means	Verified by
One list	Every exception is an entry in one list, which every check reads. Inline suppressions are turned off in the code, so that the list holds them all. A run without the list is not a pass.	Automated.
An entry	An entry names the check as the report prints it, with the file and line, such as `Wconversion:*/src/example.c:42`.	Automated.
Reason	An entry says why the exception is safe, who approved it and when.	A person.
Approval	A new deviation needs a person's approval, asked for before it is added.	A person.
Mandatory	A Mandatory rule is never deviated from.	A person. *Only the comment at the top of the list says so.*
Permits	An entry may cite a permit, and still needs its own reason and approver.	A person.
Errors	An error with no check named, such as a syntax error, can't be deviated from.	Automated.

1.3 Rules that most shape the code

These are the guidelines that decide most of how libterae's code looks. Chapter 4 has the rest.

Table 1.4 Rules that most shape the code

Rule	What it means	Verified by
D4.12, 21.3	No dynamic memory: no `malloc`, `free` or the functions like them.	Automated, for rule 21.3 and, in part, for directive 4.12.
21.6	No input and output from `<stdio.h>`.	Automated.

continues

coding-guidelines-en.tex Page 4 November 8, 2026 13:15

Table 1.4 Rules that most shape the code (*continued*)

Rule	What it means	Verified by
17.2	No recursion.	Automated, in part. *Recursion through a function pointer is a known miss.*
15.1	No `goto`.	Automated. *Labels are rejected too, which rules out `goto` altogether.*
15.5	One exit point per function.	Automated.
15.6	Braces around every `if`, `else` and loop body.	Automated.
16.4	Every `switch` has a `default`.	Automated.
D4.6	Types that state their size, such as `uint32_t`, not plain `int`.	Automated, in part.
D4.10	Every header has an include guard, named `MQTT_<FILE NAME>_H`.	Automated.
D4.14	Every value received from outside the program is checked, including every MQTT packet.	A person. *The tests and the fuzz targets try it, and can't prove it.*

One more rule shapes the interface, though no guideline asks for it. A function of the interface tests each pointer it is given before it reads or writes through any of them. A pointer may be `NULL` only where the function's documentation says it may, or where it comes with a length of zero. Anywhere else the call is refused: it changes nothing, and returns what the same call returns for an object that isn't set up. This goes beyond directive 4.14, which is about values from outside the program: it makes a caller's mistake an error instead of a crash. No tool checks it, so each such function has a test that gives it `NULL` for each pointer in turn.

1.4 Copyright and licence

libterae is offered under the GPL and under a commercial licence, so all of its code must be able to ship under both.

coding-guidelines-en.tex Page 5 November 8, 2026 13:15

Table 1.5 Copyright and licence

Rule	What it means	Verified by
Notice	Every `.c` and `.h` file starts with a comment holding its copyright notice, before anything else.	Automated, in every folder.
Whose	The notice is the right one, for the right owner.	A person. *A pass only shows that a copyright exists.*
Citing	A comment that cites a document gives its title and section, not its copyright line, which would read as the copyright of the code. It says what the document says in its own words.	A person.
Code from elsewhere	It keeps its copyright and licence notices, and is used only if its licence lets it ship under both licences. MIT and BSD code usually can. Someone else's GPL code can't, unless its copyright holder agrees.	A person. *Each folder of `src/port/` holds a `NOTICE` saying where its files came from, and their licences.*
Contributions	Before anything from another person is merged, even a small fix, they sign an agreement giving the right to relicense it.	A person.

1.5 Documentation comments

MISRA C:2012 doesn't say what needs documenting, where it goes or what it contains. It only limits what can be inside a comment, in rules 3.1 and 3.2 and directive 4.4. So these rules are the project's own. They are strict: one form of block only, everywhere. Unlike MISRA's rules, they have no deviations, and a finding can't be suppressed.

Everything named needs a block: macros, functions, typedefs, tags, enum constants, struct and union members, and objects at every scope, including local and `for` loop variables. Parameters are documented in their function's block. The only exception is the include guard, `MQTT_<FILE NAME>_H`, when it is the first thing in a header:

```
/**
 * @file base_binary_t.h
 * @brief Binary Data.
 *
 * On the wire, a 16-bit count of the data bytes comes first and the data
```

coding-guidelines-en.tex Page 6 November 8, 2026 13:15


```
 * bytes come after it.
 *
 * [V5] MQTT Version 5.0, OASIS Standard, 7 March 2019, 137 pages.
 * @li Cited: 1.5.6, p.19.
 * @li Local: the specification, kept with the project.
 * @li Source: docs.oasis-open.org/mqtt/mqtt/v5.0/os/
 */
#ifndef MQTT_BASE_BINARY_T_H
#define MQTT_BASE_BINARY_T_H

/** @brief Reference to binary data. It does not own the bytes. */
typedef struct {
    /** @brief Number of bytes in data. */
    uint16_t length;
} mqtt_binary;

/**
 * @brief Report whether two strings differ.
 * @param[in] actual String under test.
 * @param[in] expected String to compare with.
 * @return 1U if they differ, otherwise 0U.
 */
static uint32_t defs_text_differs(const char *actual, const char *expected);
```

Table 1.6 Documentation comments

Rule	What it means	Verified by
DOC-1	`/** */` is the only form, written on one line, or as `/**`, then `* text` lines, then `*/`. `/*!`, `/***`, `/**<`, `///` and `//!` all fail.	Automated.
DOC-2	A block sits directly above the first token of what it documents, at the same indent, with nothing between them. A `for` loop variable's block goes above the `for`.	Automated.
DOC-3	Everything has one block, and every block belongs to one thing. A function or object is documented on its first declaration only, and a header comes before a C file. A later declaration has no block and uses the same parameter names.	Automated.
DOC-4	A file starts with its copyright comment, then a block starting `@file` and its exact name, before any code.	Automated.
DOC-5	A block starts with one `@brief`, after `@file` in a file's own block. The only tags are `@file`, `@brief`, `@param`, `@return`, `@retval`, `@li`, and `@c` and `@p` inside text. `@li` follows a paragraph or another `@li`. There are no `\` commands, and no `@code`, which would conflict with directive 4.4.	Automated.

continues

coding-guidelines-en.tex Page 7 November 8, 2026 13:15

Table 1.6 Documentation comments (*continued*)

Rule	What it means	Verified by
DOC-6	Functions, function-like macros and function pointer typedefs have one `@param[in|out|in,out] name Text.` per parameter, in order. Functions have one `@return` if they return a value, and none otherwise. It is optional on macros.	Automated.
DOC-7	Text is ASCII, stays within column 80 (STYLE-1), and ends in a full stop, except on `@li` lines. It can't contain `\`, `<`, `&`, `#` or `%`, or `$` before a letter, and each line has balanced `"`, which a comment reader would take for a command.	Automated.
DOC-8	Other comments are `/* */`, and only for the copyright, on `#endif`, in a function body, or in an initialiser. They can't be directly above something that needs a block. `//` comments are not allowed.	Automated.
DOC-9	The code has to be something the check can fully read. Anything else is a finding, never skipped.	Automated.
Truth	A comment is true, useful and up to date, and the code can be traced to what it is for, as directive 3.1 asks.	A person. *A pass only shows that each comment exists, is in the right place and is well formed.*

DOC-9 rules out some legal C, because each of these could hide something from the check:

- More than one thing per declaration, such as `int a, *b;`.
- Function pointers, except as `typedef R (*name)(params);`.
- A struct, union or enum body that isn't exactly `typedef struct { ... } name;` or `struct name { ... };`. This rules out `typedef struct x { ... } x;`, `struct { ... } x;` and nested bodies. For an opaque type, use `typedef struct x x;`, and define `struct x` in the C file.
- `struct x;` alone, and a tag that is never defined or typedef'd.
- Type bodies in `sizeof`, casts, compound literals or parameters.
- `f()`, K&R definitions, `...` and unnamed parameters.
- `#undef`, `#if 0`, `#line`, `#pragma`, `_Pragma`, `asm`, `__attribute__`, `goto`, labels, trigraphs and digraphs. Line splices are allowed only in `#define`.
- `#if` and its `#else` and `#endif` at different levels of nesting, or inside a declaration.

coding-guidelines-en.tex Page 8 November 8, 2026 13:15

- `#include` after code, of anything but a `.h` file named directly, or of a system header the check doesn't list. MISRA bans some system headers, so they aren't listed.
- Macro bodies containing `;`, `{` or `}`, a storage class, `typedef`, `struct`, `union`, `enum` or `##`, or with a type or a parameter outside parentheses. This rules out `do { ... } while (0)` macros: write a function, as directive 4.9 prefers.

1.6 Additional coding style rules

These are the additional coding style rules, adopted in part from Texas Instruments' *Recommended C Style and Coding Guidelines*. They apply to every `.c` and `.h` file in `include/`, `platform/` and `src/`, and to the tests. Like the documentation rules, they have no deviations. In the table, the second column gives the number the rule has in that document, for reference.

Table 1.7 Additional coding style rules

Rule	TI	What it means	Verified by
STYLE-1	R1, R2	No line passes column 80, comments included. A statement that doesn't fit breaks after a comma, an operator or an opening bracket: never before an operator, and never between a type and the name it declares. Adjacent string literals may go on separate lines.	Automated.
STYLE-2	R3	Indent with spaces, never tabs: 4 per level, from column 1. What is inside braces goes one level deeper than the braces, and `case` and `default` sit at the level of their `switch`. A continuation line may align with what it continues. A comment that starts a line starts at a multiple of 4.	Automated.
STYLE-3	R4	No two blank lines in a row, and no blank line at the end of a file, which ends with a newline. One blank line separates the steps of a block.	Automated. *The blank line between steps of a block is for a person.*

continues

coding-guidelines-en.tex Page 9 November 8, 2026 13:15

Table 1.7 Additional coding style rules (*continued*)

Rule	TI	What it means	Verified by
STYLE-4	R5	One space on each side of a binary operator, which includes `=`, the compound assignments and `? :`. No space after a unary operator (`!x`, `~x`, `-x`, `*p`, `&x`), around `->` and `.`, after `(` and `[` or before `)` and `]`, or between a name and its `(`. One space after a comma or a semicolon, and none before. One space between `if`, `for`, `while`, `switch` or `return` and its `(`, and none after `sizeof`.	Automated.
STYLE-5	R9	No `++` or `--`: write `x += 1U`. TI allows them alone in a statement, and libterae doesn't use them at all.	Automated.
STYLE-6	R45	A file's top-level parts come in this order: includes, macros, types, objects, function declarations, function definitions. The copyright and `@file` block come before them (DOC-4), and an include guard goes around them.	Automated.
STYLE-7	G15	A file has at most 2,000 lines. A longer one is split into files that each hold one part of the job.	Automated.
STYLE-8	G16	A function in `src/` or `platform/` has at most 50 executable lines: the lines of its body that hold code, apart from lines holding only braces and declarations that give no value.	Automated. *Tests are exempt.*

1.6.1 The rest of the additional style rules

The rest of the rules of that document are covered elsewhere, or were not adopted:

- **Covered by MISRA C:2012.** R8 (12.1), R11 (D4.4), R14 (D4.10), R19, R31, R32 and R36 (8.1 to 8.8, 17.3), R22 (11.9), R44 (15.6), G17 (15.5), G20 (9.1) and G21 (13.4). Its section 2 lists the MISRA C:2004 rules it follows. libterae follows MISRA C:2012 instead.
- **Covered by the documentation rules and the copyright notice.** R6, R7, R13, R16, R41, R46 and G10.
- **Covered by the project rules.** G1 and G18, in Chapter 2.
- **Not adopted.** The names (R17, R18, R20, R21, R34, R35, R39, R40, G6 to G9, G11 to G13), the file names (R23 to R28), brace placement (R43), comments (R12, G3, G4), the contents of doc blocks (R29, R38, R42, G14), revision histories (G2), comment density (G5), extra variables (G19), and headers (R10,

coding-guidelines-en.tex Page 10 November 8, 2026 13:15

R15, R30, R33 and R37). libterae's own rules decide these, so its names stay `lower_snake_case`, and control statements keep their opening brace on the same line.

2

Project Rules

These rules say where code goes, what it may call, how programs and tests are arranged, how protocol behaviour is taken from the specifications, and how a change is verified before it is called done.

2.1 Where code goes

The project is the library. The server and the client are tests that show it works, not products. The library is `src/` and `include/`, and `platform/` is the part that talks to the operating system.

Table 2.1 The folders

Path	Contents
`include/base_*.h`	Library: the definitions shared by the versions, with the `MQTT_` and `mqtt_` prefixes.
`include/v31_*.h`, `v311_*.h`, `v5_*.h`	Library: the definitions for one version, with `MQTT31_`, `MQTT311_` and `MQTT5_`.
`src/`	Library: the implementation, with no system calls, no allocator and no C library.
`src/port/{architecture}/`	Library: the memory functions the compiler's code calls, one folder per architecture: `aarch64`, `arm`, `armv4t`, `armv7m`, `avr`, `i386`, `m68k`, `mips`, `powerpc64`, `riscv32`, `riscv64`, `rx`, `sh`, `x86_64` and `xtensa`. Each holds a `NOTICE` saying where its files came from and their licences.

continues

coding-guidelines-en.tex Page 12 November 8, 2026 13:15

Table 2.1 The folders (*continued*)

Path	Contents
`src/port/generic/`	Library: C `memcpy`, `memmove`, `memset` and `memcmp`, the fallback for the architectures with no assembly for them, and for any processor no other folder is written for. A folder includes one of its files where it has no assembly for that function.
`platform/`	Sockets, clocks and TLS for the supported systems: POSIX, Linux, macOS and Windows, and the real-time systems FreeRTOS and Micrium's uC/OS-III. Outside the MISRA claim.
The tests	A test that checks every definition, one test per module, a server and a client built on the library, a load generator and the fuzz targets.

Table 2.2 Where code goes

Rule	What it means	Verified by
The library	All protocol policy and algorithms belong in the library. The server and the client only call its interface, and hold no decision or algorithm of their own. If either needs something the library doesn't offer, it is added to the library.	A person.
Flat folders	`src/` and `include/` have no subfolders, except `src/port/{architecture}/`.	A person.
Port folders	`src/port/{architecture}/` holds the memory functions the compiler calls, in assembly where another project's can be used and in C from `src/port/generic/` where not. What comes from another project is taken verbatim. The folders are outside the C checks.	Automated: each `port_*.S` defines only the four memory functions and names C reserves, and leaves nothing undefined. A person, for each folder's `NOTICE`.
Generic C	`src/port/generic/` has C versions of `memcpy`, `memmove`, `memset` and `memcmp`, which never call any of the four memory functions.	Automated, at every optimisation level.

continues

coding-guidelines-en.tex Page 13 November 8, 2026 13:15

Table 2.2 Where code goes (*continued*)

Rule	What it means	Verified by
Prefixes	The names in `base_` files start with `MQTT_` or `mqtt_`, and those in one version's files with `MQTT31_`, `MQTT311_` or `MQTT5_`.	A person. *`MQTT_<FILE NAME>_H` is the only include guard accepted without a block.*
Names	Names are built from the specifications' own terms, such as Packet Identifier, Fixed Header or Receive Maximum, not invented names or abbreviations.	A person.

2.2 What the code may call

Table 2.3 What the code may call

Rule	What it means	Verified by
No C library	`src/` has no system calls, no allocator and nothing from the C library: what it calls is only the compiler's helpers.	Automated: on every target, at `-O0`, `-O1`, `-O2` and `-Os`, and rule 21.3 and directive 4.12 reject the allocator. The build uses `-ffreestanding`.
One list per system	What the code takes from outside the library is only what its system's list allows, and the lists are the ones the folders of `platform/` are built for: C99 and POSIX.1-2008 for `posix/`, Linux's additions for `linux/`, macOS's for `macos/`, Winsock 2 and the Windows API for `win32/`, FreeRTOS and FreeRTOS+TCP for `freertos/`, and uC/OS-III and uC/TCP-IP for `ucos3/`. There is no exception for speed.	Automated: what each object calls is compared with its system's list, and a name is added only with the reason it is needed.

continues

coding-guidelines-en.tex Page 14 November 8, 2026 13:15

Table 2.3 What the code may call (*continued*)

Rule	What it means	Verified by
POSIX's base	Where a folder builds on POSIX, it keeps to the base of POSIX.1-2008. An optional part of POSIX, or a file POSIX doesn't name, is used only where the base has no way to do the job, and each such use is an exception stated with its reason. A system that does its own way keeps to its list: `linux/` waits with epoll in place of `poll()`, and `macos/` with kqueue.	Automated: `posix/` compiles with `_POSIX_C_SOURCE` alone, so a function POSIX lacks is undeclared, which is an error, and what it calls is compared with the list, where each exception stands with its reason.
System headers	A system header only its own system defines is added only in its folder, and only `platform/` includes the operating system's and the TLS library's headers.	Automated: a header outside the folder's list is a finding, and `src/` can't include `platform/`'s.
`platform/`	`platform/` is outside the MISRA claim, as the project's own analysis of that code is not complete.	A person: its MISRA findings are reported, and a reviewer reads them. They don't fail a run.

2.3 Programs and tests

The tests are kept in a test and validation folder of the project. They are not products: they exist to show that the library works.

Table 2.4 Programs and tests

Rule	What it means	Verified by
Guarded `main`	Each program's `main` is guarded by a macro such as `SERVER_PROGRAM`, because MISRA reads all the programs as one. A new program is also registered with the test runner.	A person.
Every definition	Every macro in `include/` is used by the test that checks the definitions, which verifies its value.	Automated: rule 2.5 reports a macro nothing uses, and the definitions test exits non-zero on a wrong value.

continues

coding-guidelines-en.tex Page 15 November 8, 2026 13:15

Table 2.4 Programs and tests (*continued*)

Rule	What it means	Verified by
Module tests	Each module has a test in the test folder, and they all pass with AddressSanitizer and UndefinedBehaviorSanitizer.	Automated, for the sanitizers. A person, for a test per module.

2.4 Protocol behaviour

Table 2.5 Protocol behaviour

Rule	What it means	Verified by
The source	Protocol behaviour is taken from the specifications, not from memory, and the file's `@file` block cites the section.	A person. *A pass only shows that the block exists.*
Every rule	Every numbered rule of MQTT 3.1.1 and 5.0, and each item of the list of what MQTT 3.1 asks for, is cited by a test, or the justification file says why no test applies.	Automated. *A citation records a belief: it can't show the test is right.*
Against others	After a change to protocol behaviour, the server and the client still work with other MQTT implementations, pass those projects' own tests, and survive fuzzing.	Automated, run one kind at a time.
Decisions	Where the specifications leave a choice open, the choice is recorded in the project's design notes.	A person.

2.5 Text from elsewhere

Table 2.6 Text from elsewhere

Rule	What it means	Verified by
Own words	MISRA's text and the specifications' text are never pasted into the project: they are described in the project's own words.	A person.

continues

coding-guidelines-en.tex Page 16 November 8, 2026 13:15

Table 2.6 Text from elsewhere (*continued*)

Rule	What it means	Verified by
One source for the rules	The description of each MISRA guideline, written in libterae's own words, is kept in one place, and every other copy is written from it. A rule is added or changed there, never in a copy.	Automated: the copies are compared with the source, and every example is run through the checks, where a violation must be reported and its conforming version must not.

2.6 Working with the checks

Table 2.7 Working with the checks

Rule	What it means	Verified by
Before done	A C change is done only when every check that applies to it ran and found nothing. A run that checked nothing is not a pass, and a check that stopped before it read the code is not a pass either.	A person: nothing runs the checks automatically.
After a change	After changing `src/` or `platform/`, the checks on the code and on what it may call are run again. After changing protocol behaviour, the tests against other implementations and the fuzzer are run. After changing comments in `include/` or `src/`, the documentation rules are checked again.	A person.
Fix the code	A finding is fixed in the code. A check is never changed to get a pass.	A person.
The checks' tests	After changing a check, its own tests are run.	A person.

3

Automated Verification

These guidelines do not depend on how they are verified, but the project develops its own external tools to check that they have been applied. Each one covers one kind of rule, and reads the code, the build or the tests. They are run by a person before a change is called done, and nothing runs them automatically.

What follows is deliberately general: the rules say what a check has to find, and the tools are free to change as long as what they find stays the same.

3.1 Kinds of automated check

The checks are of these kinds, and a rule is verified by whichever of them applies to it:

Table 3.1 Kinds of automated check

Kind	What it reads
The source text	The file as written, without preprocessing it, so every branch of a conditional is read. It finds what the text says on its own: the shape of the comments, the indentation, the line length, the order of a file's parts, the include guards and the copyright notices.
The compiled program	The code as the compiler builds it: its real types, through every typedef and macro, and every warning the project asks for, treated as an error.
Analysis of the syntax tree	The code as a compiler sees it, without running it: conversions, declarations, the preprocessor, the standard library, and the rules that need to know the whole program at once.

continues

coding-guidelines-en.tex Page 18 November 8, 2026 13:15

Table 3.1 Kinds of automated check (*continued*)

Kind	What it reads
Analysis of values and paths	What a variable can hold at a given line, and which paths through a function are reachable: unused values, uninitialised reads, out-of-bounds accesses, divisions by zero and missing returns.
What the object calls	The symbols each object file refers to, compared with the list of the system it is built for, on every target and processor supported.
The tests	The behaviour of the library when it is run: the definitions test, one test per module under the two sanitizers, the protocol rules cited by a test, the tests written by other implementations, and the fuzz targets.
The checks themselves	The checks' own tests, which show that a check reports what it should and stays silent about what it should. A check that has never been seen to fire is not counted.

3.2 Verifying a change

A C change is done only when every check that applies to it ran to the end and found nothing. A check that found nothing because it read nothing, or because it stopped before it could read the code, is not a pass.

Table 3.2 What a change is verified with

A change in	Needs the checks of
`src/` or `include/`	A check of the source text, of the build, of the syntax tree and of the values, and the test that checks the definitions.
`platform/`	The same, plus a check that what the object calls is on its system's list, for every system and on every processor.
The protocol's behaviour	The tests that cite the protocol's rules, the tests written by other implementations, and the fuzz targets, one kind of test at a time.
A check	That check's own tests.

3.3 The checks' own rules

The checks are part of the project, and follow these rules:

coding-guidelines-en.tex Page 19 November 8, 2026 13:15

Table 3.3 The checks' own rules

Rule	What it means	Verified by
One finding, one rule	A finding names the rule it breaks, so that the rule can be looked up, fixed and, if need be, deviated from.	Automated: a finding with no rule is a failure of the check.
No silent skip	Code that a check can't classify is a finding, never something it skips.	Automated.
Nothing found is not a pass	A check reports that it read nothing, and that is not a pass.	Automated.
Proven to fire	Every check is shown to fire on code that breaks the rule, and to stay silent on code that follows it.	A person, for the cases chosen. Automated, for the running of them.
One exception list	Every deviation is recorded in one list, which the checks read, and the code holds no suppression of its own.	Automated.
The code is fixed	A finding is fixed in the code, never by changing a check.	A person.

4

MISRA C:2012, Guideline by Guideline

This chapter lists all 221 guidelines of MISRA C:2012: the 2013 edition, Amendments 1 to 4 and Technical Corrigendum 2. Each row says what the guideline asks for, in libterae's own words, and how it is verified. The exact wording is MISRA's, and is in the guidelines published by MISRA.

4.1 Reading a row

Under each guideline's number is its category. The last column says how the guideline is verified:

- **Automated.** A check reports a violation and names the guideline.
- **A person.** No check covers the guideline, so a reviewer has to verify it. These are the guidelines that depend on what the program does when it runs, or on intent.

The word may carry a note:

- **In part.** A check finds some violations, and a person still reviews the rest. The guideline depends on what the program does when it runs, or on intent.
- **Amendments 3 and 4.** Verified like the others, but outside the project's claim, as Section 1.1 says.
- **Does not arise in C99.** The guideline is about a C11 feature, which C99 code can't use. The build rejects it as C99.

Chapter 5 counts these, and lists the forms of violation that a check is known to miss.

coding-guidelines-en.tex Page 22 November 8, 2026 13:15

4.2 Directives

The directives are about the code as a whole. Directive 1 is about the implementation, 2 about compilation, 3 about requirements, 4 about code design and 5 about concurrency.

Table 4.1 Directives 1.1 to 5.3

Guideline	What it asks for	Verified by
Dir 1.1 *Required*	Where the program's output depends on something the C standard leaves to the compiler, the project writes down what this compiler does.	A person.
Dir 2.1 *Required*	Every source file compiles without a single error.	Automated.
Dir 3.1 *Required*	Every piece of code is there because a documented requirement asks for it, and can be traced back to that requirement.	A person.
Dir 4.1 *Required*	The code is written so that failures at run time, such as a division by zero, an overflow, an index out of range or a null pointer, cannot happen, and checks the values that could cause one.	Automated. *In part.*
Dir 4.2 *Advisory*	Every use of assembly language is explained where it appears.	A person.
Dir 4.3 *Required*	Assembly language is kept apart from the C code, in functions or macros that hold nothing else.	Automated. *In part.*
Dir 4.4 *Advisory*	Code that is no longer wanted is deleted, not left behind in a comment.	Automated. *In part.*
Dir 4.5 *Advisory*	Two names that can be seen in the same place, and are in the same name space, do not look alike: they differ by more than case, or by more than characters such as `l` and `1` or `O` and `0`.	Automated. *In part.*
Dir 4.6 *Advisory*	Numbers use typedefs that state their size and signedness, such as `uint32_t`, instead of the basic types whose size depends on the machine.	Automated. *In part.*
Dir 4.7 *Required*	When a function reports an error, the caller tests what it reported.	Automated. *In part.*
Dir 4.8 *Advisory*	A file that only passes a pointer to a structure or union around, and never looks inside it, is not shown what the structure contains: it sees an incomplete declaration.	Automated. *In part.*

continues

coding-guidelines-en.tex Page 23 November 8, 2026 13:15

Table 4.1 Directives 1.1 to 5.3 (*continued*)

Guideline	What it asks for	Verified by
Dir 4.9 *Advisory*	Where a function would do the same job as a function-like macro, the function is used.	Automated. *In part.*
Dir 4.10 *Required*	Every header protects itself from being included twice, for example with an include guard around its whole contents.	Automated.
Dir 4.11 *Required*	A value passed to a library function is checked first, when the function's result is undefined for some values.	Automated. *In part.*
Dir 4.12 *Required*	Memory is never allocated or freed while the program runs: there is no heap.	Automated. *In part.*
Dir 4.13 *Advisory*	Functions that work on the same resource are called in the order the resource needs: opened before it is used, and not used after it is closed.	Automated. *In part.*
Dir 4.14 *Required*	Every value that comes from outside the program, such as the bytes of a received packet, is checked before it is used.	A person.
Dir 4.15 *Required*	A floating-point calculation that could produce an infinity or a NaN has its result tested before the result reaches code that is not designed to handle one.	A person. *Amendments 3 and 4.*
Dir 5.1 *Required*	Threads do not race: data that two threads use is protected, so that one cannot write it while the other reads or writes it.	Automated. *In part.* *Amendments 3 and 4.*
Dir 5.2 *Required*	Threads cannot deadlock: locks are always taken in the same order, so no two threads each wait for a lock the other holds.	A person. *Amendments 3 and 4.*
Dir 5.3 *Required*	Threads are only created while the program starts up, never later while it runs.	Automated. *In part.* *Amendments 3 and 4.*

4.3 The language standard

coding-guidelines-en.tex Page 24 November 8, 2026 13:15

Table 4.2 Rules 1.1 to 1.5

Guideline	What it asks for	Verified by
Rule 1.1 *Required*	The code is standard C: it breaks none of the language's syntax rules or constraints, and stays within the translation limits that every compiler has to support.	Automated.
Rule 1.2 *Advisory*	The code does not use a compiler's own additions to the language.	Automated.
Rule 1.3 *Required*	Nothing in the program has undefined behaviour, or unspecified behaviour that would change what it does.	Automated. *In part.*
Rule 1.4 *Required*	The optional bounds-checking functions of C11's Annex K, such as `memcpy_s`, are not used. Amendments 3 and 4 took the other C11 features off this rule and gave them guidelines of their own.	Automated.
Rule 1.5 *Required*	Features that the C standard marks as obsolescent are not used, such as a storage-class specifier after the type, a function declared with empty parentheses, or a definition that lists its parameters' types separately.	Automated. *Amendments 3 and 4.*

4.4 Unused and dead code

Table 4.3 Rules 2.1 to 2.8

Guideline	What it asks for	Verified by
Rule 2.1 *Required*	There is no code that can never run.	Automated. *In part.*
Rule 2.2 *Required*	There is no dead code: nothing runs whose removal would leave the program behaving the same.	Automated. *In part.*
Rule 2.3 *Advisory*	Every type that is declared is used somewhere.	Automated.
Rule 2.4 *Advisory*	Every tag of a structure, union or enumeration that is declared is used somewhere.	Automated.

continues

coding-guidelines-en.tex Page 25 November 8, 2026 13:15

Table 4.3 Rules 2.1 to 2.8 (*continued*)

Guideline	What it asks for	Verified by
Rule 2.5 *Advisory*	Every macro that is defined is used somewhere.	Automated.
Rule 2.6 *Advisory*	A function has no label that nothing jumps to.	Automated.
Rule 2.7 *Advisory*	A function uses every one of its parameters.	Automated.
Rule 2.8 *Advisory*	Every object that is defined is used somewhere.	Automated. *Amendments 3 and 4.*

4.5 Comments

Table 4.4 Rules 3.1 to 3.2

Guideline	What it asks for	Verified by
Rule 3.1 *Required*	A comment does not contain the start of another comment, `/*` or `//`, apart from a web address such as `https://example.org`.	Automated.
Rule 3.2 *Required*	A `//` comment does not end in a backslash, which would join the next line to the comment.	Automated.

4.6 Characters and escape sequences

Table 4.5 Rules 4.1 to 4.2

Guideline	What it asks for	Verified by
Rule 4.1 *Required*	An octal or hexadecimal escape sequence in a string or character constant is followed directly by another escape sequence or by the end of the literal, so it cannot run on into the characters after it.	Automated.
Rule 4.2 *Advisory*	Trigraphs, such as `??=` for `#`, are not used.	Automated.

coding-guidelines-en.tex Page 26 November 8, 2026 13:15

4.7 Identifiers

Table 4.6 Rules 5.1 to 5.9

Guideline	What it asks for	Verified by
Rule 5.1 *Required*	External names differ within their first 31 characters, the most that C99 promises a linker will compare.	Automated.
Rule 5.2 *Required*	Two names declared in the same scope and name space differ within their first 63 characters, the most that C99 promises a compiler will compare.	Automated.
Rule 5.3 *Required*	A name declared inside a block does not hide a name of the same spelling from an enclosing scope.	Automated.
Rule 5.4 *Required*	Macro names differ from each other within their first 63 characters, and from the names of macro parameters; the parameters of one macro differ from each other.	Automated.
Rule 5.5 *Required*	A name used for a macro is not also used for anything else, such as an object or a function.	Automated.
Rule 5.6 *Required*	A `typedef` name is used for that `typedef` only, and for nothing else anywhere in the program.	Automated.
Rule 5.7 *Required*	The tag of a structure, union or enumeration is used for that tag only, and for nothing else anywhere in the program.	Automated.
Rule 5.8 *Required*	The name of an object or function with external linkage is used for nothing else in the whole program, not even a local in another file.	Automated.
Rule 5.9 *Advisory*	The name of an object or function with internal linkage is used for nothing else in the program, not even in another file.	Automated.

4.8 Bit-fields

coding-guidelines-en.tex Page 27 November 8, 2026 13:15

Table 4.7 Rules 6.1 to 6.3

Guideline	What it asks for	Verified by
Rule 6.1 *Required*	A bit-field is declared as `unsigned int`, `signed int` or `_Bool`, or as another explicitly signed or unsigned integer type that the compiler documents, never as plain `int` or an enumeration.	Automated.
Rule 6.2 *Required*	A named bit-field one bit wide is not of a signed type.	Automated.
Rule 6.3 *Required*	A member of a union is not a bit-field.	Automated. *Amendments 3 and 4.*

4.9 Literals and constants

Table 4.8 Rules 7.1 to 7.6

Guideline	What it asks for	Verified by
Rule 7.1 *Required*	Octal constants, which start with `0`, are not used, apart from `0` itself.	Automated.
Rule 7.2 *Required*	An integer constant whose type is unsigned is written with a `u` or `U` suffix.	Automated.
Rule 7.3 *Required*	A constant's suffix uses a capital `L`, never a lowercase `l`, which looks like the digit `1`.	Automated.
Rule 7.4 *Required*	A string literal is only assigned to a pointer to `const char`, so it cannot be written through.	Automated.
Rule 7.5 *Mandatory*	The argument of an integer constant macro such as `UINT32_C` is a plain integer constant with no suffix, and fits the width the macro's name gives.	Automated. *Amendments 3 and 4.*
Rule 7.6 *Required*	The integer constant macros for types narrower than `int`, `INT8_C`, `INT16_C`, `UINT8_C` and `UINT16_C`, are not used: what they give has the type of `int`.	Automated. *Amendments 3 and 4.*

4.10 Declarations and definitions

coding-guidelines-en.tex Page 28 November 8, 2026 13:15

Table 4.9 Rules 8.1 to 8.17

Guideline	What it asks for	Verified by
Rule 8.1 *Required*	Every declaration states its type: none relies on a missing type meaning `int`.	Automated.
Rule 8.2 *Required*	A function's type is written as a prototype, and every parameter in it has a name.	Automated.
Rule 8.3 *Required*	Every declaration of an object or function uses the same parameter names and the same type names and qualifiers as the others.	Automated.
Rule 8.4 *Required*	An object or function with external linkage has a compatible declaration in view where it is defined, normally from its header.	Automated.
Rule 8.5 *Required*	An external object or function is declared once, in one file, normally a header.	Automated.
Rule 8.6 *Required*	An identifier with external linkage has exactly one definition in the whole program.	Automated.
Rule 8.7 *Advisory*	A function or object that only one file uses is not given external linkage: it is made `static` in that file.	Automated.
Rule 8.8 *Required*	Every declaration of an object or function with internal linkage says `static`, not only the first.	Automated.
Rule 8.9 *Advisory*	An object that only one function uses is declared inside that function, not at file scope.	Automated.
Rule 8.10 *Required*	An `inline` function is declared `static`.	Automated.
Rule 8.11 *Advisory*	An array with external linkage is declared with its size stated.	Automated.
Rule 8.12 *Required*	In an enumeration, a constant whose value is implied, by having no = of its own, does not end up with the same value as another constant.	Automated.
Rule 8.13 *Advisory*	A pointer points to a `const`-qualified type whenever nothing is written through it.	Automated. *In part.*
Rule 8.14 *Required*	The `restrict` qualifier is not used.	Automated.
Rule 8.15 *Required*	When any declaration of an object gives it an alignment, every declaration of that object gives the same alignment, written the same way.	Does not arise in C99. *Amendments 3 and 4.*

continues

coding-guidelines-en.tex Page 29 November 8, 2026 13:15

Table 4.9 Rules 8.1 to 8.17 (*continued*)

Guideline	What it asks for	Verified by
Rule 8.16 *Advisory*	An object's declaration does not ask for an alignment of zero.	Does not arise in C99. *Amendments 3 and 4.*
Rule 8.17 *Advisory*	An object's declaration has at most one alignment specifier.	Does not arise in C99. *Amendments 3 and 4.*

4.11 Initialisation

Table 4.10 Rules 9.1 to 9.7

Guideline	What it asks for	Verified by
Rule 9.1 *Mandatory*	An automatic object is given a value before anything reads it.	Automated. *In part.*
Rule 9.2 *Required*	The initializer of an array, structure or union is in braces, with nested braces for each array, structure or union inside it.	Automated.
Rule 9.3 *Required*	An array is initialised in full, or with `{ 0 }` to set all of it to zero, or with designated initializers only.	Automated.
Rule 9.4 *Required*	No element of an object is given a value twice in one initializer.	Automated.
Rule 9.5 *Required*	An array initialised with designators has its size stated, so that the size does not depend on the highest designator.	Automated.
Rule 9.6 *Required*	An initializer that uses a chained designator, such as `.header.type`, gives a designator to every value it lists.	Automated. *Amendments 3 and 4.*
Rule 9.7 *Mandatory*	An atomic object is given its first value, in its declaration or with `atomic_init`, before anything else uses it.	Does not arise in C99. *Amendments 3 and 4.*

4.12 Essential types

coding-guidelines-en.tex Page 30 November 8, 2026 13:15

Table 4.11 Rules 10.1 to 10.8

Guideline	What it asks for	Verified by
Rule 10.1 *Required*	Each operand has an essential type that suits its operator: no arithmetic on Boolean or character values, no bitwise or shift operations on signed values, no Boolean operators on numbers.	Automated.
Rule 10.2 *Required*	A character is only added to a number, or has a number or another character taken from it: one character added to another, or a character taken from a number, has no meaning.	Automated.
Rule 10.3 *Required*	A value is not assigned to an object of a narrower essential type, or of a different essential type category, such as signed to unsigned. The same holds for arguments and return values.	Automated.
Rule 10.4 *Required*	The two operands of an operator that brings them to a common type, such as `+` or `<`, are of the same essential type category, for example both unsigned.	Automated.
Rule 10.5 *Advisory*	A cast does not convert a value to an essential type it does not belong in, such as a number to `bool`, or a number to an enumeration.	Automated.
Rule 10.6 *Required*	The result of an operation is not assigned straight to an object of a wider essential type: the operands are widened first, so the operation itself happens in the wider type.	Automated.
Rule 10.7 *Required*	When the result of an operation is an operand of a further operator, the other operand is not of a wider essential type: the inner operation is widened first.	Automated.
Rule 10.8 *Required*	The result of an operation is not cast to a wider essential type, or to a different essential type category: the operands are cast instead.	Automated.

4.13 Pointer conversions

coding-guidelines-en.tex Page 31 November 8, 2026 13:15

Table 4.12 Rules 11.1 to 11.10

Guideline	What it asks for	Verified by
Rule 11.1 *Required*	A pointer to a function is not converted to any other type, and nothing else is converted to a pointer to a function.	Automated.
Rule 11.2 *Required*	A pointer to an incomplete type, one whose contents this file cannot see, is not converted to or from any other type.	Automated.
Rule 11.3 *Required*	A pointer to one object type is not converted to a pointer to a different object type, except to a pointer to a character type for reading the object's bytes.	Automated.
Rule 11.4 *Advisory*	A pointer to an object is not converted to an integer, and an integer is not converted to a pointer to an object.	Automated.
Rule 11.5 *Advisory*	A pointer to `void` is not converted to a pointer to an object type.	Automated.
Rule 11.6 *Required*	A cast does not convert between a pointer to `void` and an arithmetic type.	Automated.
Rule 11.7 *Required*	A cast does not convert between a pointer to an object and an arithmetic type that is not an integer, such as `bool`, a character or a floating-point type.	Automated.
Rule 11.8 *Required*	No conversion, whether by a cast or implicit, takes away the `const`, `volatile` or `_Atomic` qualifier of what a pointer points to.	Automated.
Rule 11.9 *Required*	A null pointer is written as `NULL`, never as the integer constant `0`.	Automated.
Rule 11.10 *Required*	The `_Atomic` qualifier is not applied to `void`.	Does not arise in C99. *Amendments 3 and 4.*

4.14 Expressions

coding-guidelines-en.tex Page 32 November 8, 2026 13:15

Table 4.13 Rules 12.1 to 12.6

Guideline	What it asks for	Verified by
Rule 12.1 *Advisory*	Where an expression mixes operators, parentheses show the order they apply in, rather than leaving the reader to recall C's precedence table.	Automated.
Rule 12.2 *Required*	The amount a value is shifted by is at least zero and less than the width, in bits, of the essential type of the value being shifted.	Automated. *In part.*
Rule 12.3 *Advisory*	The comma operator is not used.	Automated.
Rule 12.4 *Advisory*	Working out a constant expression does not make an unsigned value wrap around.	Automated.
Rule 12.5 *Mandatory*	`sizeof` is not applied to a parameter declared as an array, because such a parameter is really a pointer and `sizeof` gives the size of the pointer.	Automated.
Rule 12.6 *Required*	A member of an atomic structure or union is not reached directly with `.` or `->`: the whole object is read or written, with the atomic functions or with `=`.	Does not arise in C99. *Amendments 3 and 4.*

4.15 Side effects

Table 4.14 Rules 13.1 to 13.6

Guideline	What it asks for	Verified by
Rule 13.1 *Required*	The values in an initializer list have no side effects that last, such as calls that change state, because C does not fix the order they are worked out in.	Automated. *In part.*
Rule 13.2 *Required*	An expression gives the same value, and leaves the same lasting effects, in every order C allows its parts to be worked out in, and however the threads of the program interleave.	Automated. *In part.*
Rule 13.3 *Advisory*	An expression that uses `++` or `--` has no other side effect, such as an assignment or a call that changes state.	Automated.

continues

coding-guidelines-en.tex Page 33 November 8, 2026 13:15

Table 4.14 Rules 13.1 to 13.6 (*continued*)

Guideline	What it asks for	Verified by
Rule 13.4 *Advisory*	The result of an assignment is not used as a value, as in a condition or in another assignment.	Automated.
Rule 13.5 *Required*	The right-hand operand of `&&` or `||` has no side effects that last, because it is only worked out when the left-hand operand does not already decide the result.	Automated. *In part.*
Rule 13.6 *Required*	The operand of `sizeof` contains nothing that could have a side effect, because the operand is not worked out at all.	Automated.

4.16 Controlling expressions

Table 4.15 Rules 14.1 to 14.4

Guideline	What it asks for	Verified by
Rule 14.1 *Required*	A loop counter does not have an essentially floating-point type.	Automated. *In part.*
Rule 14.2 *Required*	A `for` loop has the simple form: one loop counter, set in the first clause, tested in the second with no side effects, changed only in the third clause and never in the body.	Automated. *In part.*
Rule 14.3 *Required*	A controlling expression, such as the condition of an `if`, is not always true or always false, apart from the forms allowed for a loop that runs forever.	Automated. *In part.*
Rule 14.4 *Required*	The condition of an `if`, `while`, `do` or `for` has an essentially Boolean type: a number is compared rather than tested as true or false.	Automated.

4.17 Control flow

coding-guidelines-en.tex Page 34 November 8, 2026 13:15

Table 4.16 Rules 15.1 to 15.7

Guideline	What it asks for	Verified by
Rule 15.1 *Advisory*	`goto` is not used.	Automated.
Rule 15.2 *Required*	A `goto` only jumps forward, to a label later in the same function. Rule 15.1 advises against `goto` altogether; this rule applies where it is used anyway.	Automated.
Rule 15.3 *Required*	A `goto` only jumps to a label in its own block or in a block that encloses it, never into another block. Rule 15.1 advises against `goto` altogether; this rule applies where it is used anyway.	Automated.
Rule 15.4 *Advisory*	A loop is ended by at most one `break` or `goto`.	Automated.
Rule 15.5 *Advisory*	A function has one exit, at its end.	Automated.
Rule 15.6 *Required*	The body of every `if`, `else`, `while`, `do`, `for` and `switch` is a compound statement, in braces; only an `else` may be followed directly by another `if`.	Automated.
Rule 15.7 *Required*	A chain of `if` and `else if` ends with a plain `else`.	Automated.

4.18 Switch statements

Table 4.17 Rules 16.1 to 16.7

Guideline	What it asks for	Verified by
Rule 16.1 *Required*	A `switch` statement has the restricted shape MISRA describes: its body holds only `switch` clauses, each starting with `case` or `default` labels and ending with `break`, and nothing comes before the first label.	Automated.
Rule 16.2 *Required*	A `case` or `default` label is only placed directly in the body of its `switch`, never inside a block nested in it.	Automated.

continues

coding-guidelines-en.tex Page 35 November 8, 2026 13:15

Table 4.17 Rules 16.1 to 16.7 (*continued*)

Guideline	What it asks for	Verified by
Rule 16.3 *Required*	Every `switch` clause ends with a `break` that always runs, so control never falls through into the next clause.	Automated.
Rule 16.4 *Required*	Every `switch` statement has a `default` label.	Automated.
Rule 16.5 *Required*	The `default` label is either the first or the last label of its `switch`.	Automated.
Rule 16.6 *Required*	Every `switch` statement has at least two `switch` clauses; one with less is written as an `if`.	Automated.
Rule 16.7 *Required*	The controlling expression of a `switch` does not have an essentially Boolean type.	Automated.

4.19 Functions

Table 4.18 Rules 17.1 to 17.13

Guideline	What it asks for	Verified by
Rule 17.1 *Required*	Functions do not take a variable number of arguments: nothing from `<stdarg.h>` is used.	Automated.
Rule 17.2 *Required*	No function calls itself, either directly or through other functions.	Automated. *In part.*
Rule 17.3 *Mandatory*	Every function is declared before it is called, so the compiler never has to guess a declaration for it.	Automated.
Rule 17.4 *Mandatory*	Every way out of a function that returns a value goes through a `return` statement that gives the value.	Automated.
Rule 17.5 *Required*	When a parameter is declared as an array of a given size, every argument passed for it is an array with at least that many elements.	Automated. *In part.*
Rule 17.6 *Mandatory*	An array parameter is not declared with `static` inside its brackets.	Automated.

continues

coding-guidelines-en.tex Page 36 November 8, 2026 13:15

Table 4.18 Rules 17.1 to 17.13 (*continued*)

Guideline	What it asks for	Verified by
Rule 17.7 *Required*	The value a function returns is used, or the call is cast to `void` to show it is deliberately thrown away.	Automated.
Rule 17.8 *Advisory*	A function does not change its own parameters; it copies one into a local first.	Automated. *In part.*
Rule 17.9 *Mandatory*	A function declared `_Noreturn` never returns to its caller, by any path.	Does not arise in C99. *Amendments 3 and 4.*
Rule 17.10 *Required*	A function declared `_Noreturn` has a `void` return type.	Does not arise in C99. *Amendments 3 and 4.*
Rule 17.11 *Advisory*	A function that can never return is declared `_Noreturn`.	Does not arise in C99. *Amendments 3 and 4.*
Rule 17.12 *Advisory*	A function's name is only used to call it, with its argument list, or with a `&` in front to take its address.	Automated. *Amendments 3 and 4.*
Rule 17.13 *Required*	A function type is not given a type qualifier such as `const` or `volatile`; a qualifier on the return type or on a pointer to the function is fine.	Automated.

4.20 Pointers and arrays

Table 4.19 Rules 18.1 to 18.10

Guideline	What it asks for	Verified by
Rule 18.1 *Required*	Pointer arithmetic and array indexing stay within the array the pointer started in, or at most one element past its end.	Automated. *In part.*
Rule 18.2 *Required*	One pointer is only subtracted from another when both point into the same array.	Automated. *In part.*
Rule 18.3 *Required*	The operators `<`, `<=`, `>` and `>=` only compare pointers that point into the same object.	Automated. *In part.*
Rule 18.4 *Advisory*	`+`, `-`, `+=` and `-=` are not used on pointers: an element is reached by indexing instead.	Automated.
Rule 18.5 *Advisory*	A declaration has at most two levels of pointer.	Automated.

continues

coding-guidelines-en.tex Page 37 November 8, 2026 13:15

Table 4.19 Rules 18.1 to 18.10 (*continued*)

Guideline	What it asks for	Verified by
Rule 18.6 *Required*	The address of an automatic or thread-local object is not kept anywhere that outlives the object, such as a static pointer or a return value.	Automated. *In part.*
Rule 18.7 *Required*	A structure does not end with a flexible array member, one declared with empty brackets.	Automated.
Rule 18.8 *Required*	Variable-length arrays, whose size is only known at run time, are not used.	Automated.
Rule 18.9 *Required*	An array inside a temporary object, such as a structure returned by a function, is not turned into a pointer: the structure is stored in a named object first.	Automated. *Amendments 3 and 4.*
Rule 18.10 *Mandatory*	A pointer to a variable-length array type, whose size is only known at run time, is not used.	Automated. *Amendments 3 and 4.*

4.21 Overlapping storage

Table 4.20 Rules 19.1 to 19.2

Guideline	What it asks for	Verified by
Rule 19.1 *Mandatory*	An object is not assigned or copied to another object that overlaps it.	Automated. *In part.*
Rule 19.2 *Advisory*	The `union` keyword is not used.	Automated.

4.22 The preprocessor

Table 4.21 Rules 20.1 to 20.14

Guideline	What it asks for	Verified by
Rule 20.1 *Advisory*	An `#include` comes before any code in the file: only other preprocessor directives and comments come before it.	Automated.

continues

coding-guidelines-en.tex Page 38 November 8, 2026 13:15

Table 4.21 Rules 20.1 to 20.14 (*continued*)

Guideline	What it asks for	Verified by
Rule 20.2 *Required*	A header's name in an `#include` has no `'`, `"` or `\` characters, and no `/*` or `//`.	Automated.
Rule 20.3 *Required*	An `#include` is followed by a header name in `<>` or `""`, and nothing else, either as written or after macros are expanded.	Automated.
Rule 20.4 *Required*	No macro has the same name as a keyword.	Automated.
Rule 20.5 *Advisory*	`#undef` is not used.	Automated.
Rule 20.6 *Required*	An argument of a macro contains no line that looks like a preprocessor directive.	Automated.
Rule 20.7 *Required*	Wherever a macro parameter is expanded into an expression, it is in parentheses, unless it is the operand of `#` or `##`.	Automated.
Rule 20.8 *Required*	The condition of an `#if` or `#elif` comes out as `0` or `1`.	Automated.
Rule 20.9 *Required*	Every name in the condition of an `#if` or `#elif` is a defined macro, apart from the operand of `defined`, so that a missing definition is not silently read as `0`.	Automated.
Rule 20.10 *Advisory*	The `#` and `##` operators of the preprocessor are not used.	Automated.
Rule 20.11 *Required*	A macro parameter that follows `#` is not then followed by `##`.	Automated.
Rule 20.12 *Required*	A macro parameter that is an operand of `#` or `##` is used nowhere else in the macro, when the argument is itself a macro that would be expanded there.	Automated.
Rule 20.13 *Required*	A line that starts with `#` is a real preprocessor directive, even in a branch of an `#if` that is skipped.	Automated.
Rule 20.14 *Required*	An `#else`, `#elif` or `#endif` is in the same file as the `#if`, `#ifdef` or `#ifndef` it belongs to.	Automated.

4.23 The standard library

coding-guidelines-en.tex Page 39 November 8, 2026 13:15

Table 4.22 Rules 21.1 to 21.26

Guideline	What it asks for	Verified by
Rule 21.1 *Required*	`#define` and `#undef` are not used on a name that C reserves: a standard macro or library name, or one that starts with an underscore and a capital letter or with two underscores.	Automated.
Rule 21.2 *Required*	No identifier is declared with a name that C reserves, such as the name of a standard library function or a name that starts with an underscore at file scope.	Automated.
Rule 21.3 *Required*	The allocation functions of `<stdlib.h>`, such as `malloc`, `calloc`, `realloc` and `free`, are not used.	Automated.
Rule 21.4 *Required*	Nothing from `<setjmp.h>`, such as `setjmp` and `longjmp`, is used.	Automated.
Rule 21.5 *Required*	Nothing from `<signal.h>` is used.	Automated.
Rule 21.6 *Required*	The input and output functions of the standard library, such as those of `<stdio.h>`, are not used.	Automated.
Rule 21.7 *Required*	`atof`, `atoi`, `atol` and `atoll` are not used, because they cannot report text that is not a number.	Automated.
Rule 21.8 *Required*	The functions of `<stdlib.h>` that end the program, `abort`, `exit`, `_Exit` and `quick_exit`, are not used.	Automated.
Rule 21.9 *Required*	`bsearch` and `qsort` from `<stdlib.h>` are not used.	Automated.
Rule 21.10 *Required*	The time and date functions of the standard library, such as those of `<time.h>`, are not used.	Automated.
Rule 21.11 *Advisory*	`<tgmath.h>` is not included. Amendment 3 made this rule advisory and gave the type-generic macros rules of their own, 21.22 and 21.23.	Automated.
Rule 21.12 *Required*	Nothing from `<fenv.h>`, the floating-point environment, is used. Before Amendment 3 the rule only covered its exception flags, and was advisory.	Automated.

continues

coding-guidelines-en.tex Page 40 November 8, 2026 13:15

Table 4.22 Rules 21.1 to 21.26 (*continued*)

Guideline	What it asks for	Verified by
Rule 21.13 *Mandatory*	A value passed to a function of `<ctype.h>` is either `EOF` or fits in an `unsigned char`, so a plain `char` is converted to `unsigned char` first.	Automated. *In part.*
Rule 21.14 *Required*	`memcmp` is not used to compare strings that end in a null character, because it compares the bytes after the end too.	Automated. *In part.*
Rule 21.15 *Required*	The two pointers given to `memcpy`, `memmove` or `memcmp` point to the same type, apart from `const` and `volatile`.	Automated.
Rule 21.16 *Required*	`memcmp` only compares objects of pointer, signed, unsigned, Boolean or enumeration type: never structures, which can hold padding, nor floating-point values or strings.	Automated.
Rule 21.17 *Mandatory*	The string functions of `<string.h>` never read or write past the end of the objects their pointers point to.	Automated. *In part.*
Rule 21.18 *Mandatory*	A size given to a function of `<string.h>` is no larger than the objects it applies to.	Automated. *In part.*
Rule 21.19 *Mandatory*	What `localeconv`, `getenv`, `setlocale` and `strerror` return is treated as pointing to `const`: it is never written through.	Automated. *In part.*
Rule 21.20 *Mandatory*	A pointer returned by `asctime`, `ctime`, `gmtime`, `localtime`, `localeconv`, `getenv`, `setlocale` or `strerror` is not used after the same function, or one that shares its storage, is called again.	Automated. *In part.*
Rule 21.21 *Required*	`system` from `<stdlib.h>` is not used.	Automated.
Rule 21.22 *Mandatory*	Each operand given to a type-generic macro of `<tgmath.h>` is a signed, unsigned or floating-point number, never a character, a Boolean or a pointer, and a complex number only where the macro has a complex form.	Implied by rule 21.11.

continues

coding-guidelines-en.tex Page 41 November 8, 2026 13:15

Table 4.22 Rules 21.1 to 21.26 (*continued*)

Guideline	What it asks for	Verified by
Rule 21.23 *Required*	Every operand given to a type-generic macro of `<tgmath.h>` that takes more than one has the same standard type, once integer promotion is done.	Implied by rule 21.11.
Rule 21.24 *Required*	`rand` and `srand` from `<stdlib.h>` are not used.	Automated. *Amendments 3 and 4.*
Rule 21.25 *Required*	Every atomic operation and fence uses the sequentially consistent memory order, `memory_order_seq_cst`, which the functions without `_explicit` use anyway.	Does not arise in C99. *Amendments 3 and 4.*
Rule 21.26 *Required*	`mtx_timedlock` is only called on a mutex created to support timeouts, with `mtx_timed`.	Does not arise in C99. *Amendments 3 and 4.*

4.24 Resources and threads

Table 4.23 Rules 22.1 to 22.20

Guideline	What it asks for	Verified by
Rule 22.1 *Required*	Every resource obtained from the standard library, such as memory or an open file, is released explicitly, on every path.	Automated. *In part.*
Rule 22.2 *Mandatory*	Only memory that a standard library function allocated is freed, and only once.	Automated. *In part.*
Rule 22.3 *Required*	A file is not open for reading and for writing at the same time, through two different streams.	Automated. *In part.*
Rule 22.4 *Mandatory*	Nothing is written to a stream that was opened for reading only.	Automated. *In part.*
Rule 22.5 *Mandatory*	A pointer to `FILE` is never dereferenced: the `FILE` object is only handled through the library's functions.	Automated.
Rule 22.6 *Mandatory*	A pointer to `FILE` is not used once its stream has been closed.	Automated. *In part.*

continues

coding-guidelines-en.tex Page 42 November 8, 2026 13:15

Table 4.23 Rules 22.1 to 22.20 (*continued*)

Guideline	What it asks for	Verified by
Rule 22.7 *Required*	`EOF` is only compared with the unchanged value returned by a library function that can return `EOF`, never with that value after it has been converted, for example to `char`.	Automated.
Rule 22.8 *Required*	`errno` is set to zero just before a call to a library function that reports errors through `errno`.	Automated. *In part.*
Rule 22.9 *Required*	After a call to a library function that reports errors through `errno`, `errno` is tested against zero.	Automated. *In part.*
Rule 22.10 *Required*	`errno` is only tested straight after a call to a library function that reports errors through `errno`, with no other call in between.	Automated. *In part.*
Rule 22.11 *Required*	A thread that has been joined or detached is never joined or detached again.	Does not arise in C99. *Amendments 3 and 4.*
Rule 22.12 *Mandatory*	Threads, mutexes, condition variables and thread-specific storage are only used through the standard library functions made for them, never copied or read directly.	Does not arise in C99. *Amendments 3 and 4.*
Rule 22.13 *Required*	Threads, mutexes, condition variables and thread-specific storage are not automatic or thread-local objects: they have static storage, or live in allocated memory.	Does not arise in C99. *Amendments 3 and 4.*
Rule 22.14 *Mandatory*	A mutex is initialised with `mtx_init`, and a condition variable with `cnd_init`, before anything uses it, and `mtx_init` is given one of the mutex types C defines.	Does not arise in C99. *Amendments 3 and 4.*
Rule 22.15 *Required*	A mutex, condition variable or thread-specific storage key is only destroyed once every thread that uses it has ended.	Does not arise in C99. *Amendments 3 and 4.*
Rule 22.16 *Required*	A thread that locks a mutex unlocks it again itself, on every path, before the thread ends.	Does not arise in C99. *Amendments 3 and 4.*
Rule 22.17 *Required*	A thread only unlocks a mutex it has locked, and only waits on a condition variable with a mutex it has locked.	Does not arise in C99. *Amendments 3 and 4.*
Rule 22.18 *Required*	A thread does not lock again a mutex it already holds, unless the mutex was created recursive.	Does not arise in C99. *Amendments 3 and 4.*

continues

coding-guidelines-en.tex Page 43 November 8, 2026 13:15

Table 4.23 Rules 22.1 to 22.20 (*continued*)

Guideline	What it asks for	Verified by
Rule 22.19 *Required*	A condition variable is only ever used with one mutex.	Does not arise in C99. *Amendments 3 and 4.*
Rule 22.20 *Mandatory*	A thread-specific storage key is created with `tss_create` before anything uses it.	Does not arise in C99. *Amendments 3 and 4.*

4.25 Generic selections

Table 4.24 Rules 23.1 to 23.8

Guideline	What it asks for	Verified by
Rule 23.1 *Advisory*	A generic selection, `_Generic`, only appears in the expansion of a function-like macro, applied to one of the macro's arguments.	Does not arise in C99. *Amendments 3 and 4.*
Rule 23.2 *Required*	A generic selection that is not expanded from a macro has no side effects, not even a call, in its controlling expression.	Does not arise in C99. *Amendments 3 and 4.*
Rule 23.3 *Advisory*	A generic selection names at least one type, besides `default`.	Does not arise in C99. *Amendments 3 and 4.*
Rule 23.4 *Required*	The types a generic selection lists can match: none is `const`, `volatile` or atomic, an array, a function, or a structure or union defined in place.	Does not arise in C99. *Amendments 3 and 4.*
Rule 23.5 *Advisory*	A generic selection does not depend on a pointer conversion that would happen implicitly elsewhere: a pointer that could have been converted to a listed type does not fall through to `default`.	Does not arise in C99. *Amendments 3 and 4.*
Rule 23.6 *Required*	The controlling expression of a generic selection has an essential type that is the same as its standard type, so a `uint8_t` sum, which C makes an `int`, is cast back first.	Does not arise in C99. *Amendments 3 and 4.*
Rule 23.7 *Advisory*	A macro built on a generic selection works out its argument exactly once, whichever type is selected.	Does not arise in C99. *Amendments 3 and 4.*
Rule 23.8 *Required*	If a generic selection has a `default` association, it is either the first or the last.	Does not arise in C99. *Amendments 3 and 4.*

5

Coverage at a Glance

This chapter counts what the earlier chapters say rule by rule: how the guidelines are verified, what is left for a person, and the forms of violation that a check is known to miss.

5.1 Guidelines by how they are verified

The counts come from Chapter 4, one row at a time. They are what the project claims, not a measurement of anything else.

Table 5.1 Guidelines by how they are verified

Verified by	Guidelines
A check, fully	137
A check, in part	47
Implied by another rule	2
Does not arise in C99	29
A person only	6
All guidelines	221

A guideline verified in part is counted once, under *in part* alone, and a guideline that does not arise in C99 is not counted as verified by anything. Of the 221, then, a check covers 184, and a person has to verify the other 6.

coding-guidelines-en.tex Page 46 November 8, 2026 13:15

5.2 Known misses

Every check is shown to fire on code that breaks a rule and to stay silent on code that follows it. Some forms of violation are known to escape, and they are listed here so that a reviewer knows where to look. A rule verified in part is, by definition, one of them.

Table 5.2 Known misses

Guideline	Form of violation	Why it is missed
Rule 2.2	a defensive first value that is overwritten	A constant first value is left alone on purpose by every analysis here: it is a common defensive habit.
Rule 8.7	a definition in a header, used in one file only	A declaration in a header counts as a use in another file, so a definition in a header is never reported.
Rule 17.2	recursion through a pointer	Seeing this needs the values a function pointer can hold, and the analysis follows direct calls only.
Rule 17.5	an object too small, reached through a local pointer	Knowing what a pointer holds at a call takes an analysis of values that no check here does.
Rule 17.8	a parameter whose address is taken and written through	Only an analysis of values could see that the callee writes through the pointer.
Dir 4.8	a structure declared in a header, never looked inside	A header's structure can have users outside the code being read, so nothing here can show it could be hidden.
Rule 20.7	a stringized argument or a call site's parentheses	The definition of the macro is reported even when every use puts brackets around its arguments, so compliant code is reported too.

5.3 What a person reviews

No check. A person has to review directives 1.1, 3.1, 4.2, 4.14, 4.15 and 5.2, as no check covers them.

In part. A check finds some violations of rules 1.3, 2.1, 2.2, 8.13, 9.1, 12.2, 13.1, 13.2, 13.5, 14.1, 14.2, 14.3, 17.2, 17.5, 17.8, 18.1, 18.2, 18.3, 18.6, 19.1, 21.13, 21.14, 21.17, 21.18, 21.19, 21.20, 22.1, 22.2, 22.3, 22.4, 22.6, 22.8, 22.9 and 22.10, and of directives 4.1, 4.3, 4.4, 4.5, 4.6, 4.7, 4.8, 4.9, 4.11, 4.12, 4.13, 5.1 and 5.3, and a person

coding-guidelines-en.tex Page 47 November 8, 2026 13:15

still reviews them.

5.4 What only a person verifies

No check covers these rules from Chapters 1 and 2. A review has to:

- **Deviations.** Each entry's reason and approver, the approval itself, and that no Mandatory rule is deviated from.
- **Copyright.** That a notice is the right one, that a cited document is cited by title and section, that code from elsewhere can ship under both licences, and that contributors signed an agreement.
- **Comments.** That each comment is true and up to date.
- **Layout.** The blank line between the steps of a block, and that the code reads well.
- **Where code goes.** That policy is in the library, that `src/` and `include/` stay flat, the `NOTICE` of each port folder, the identifier prefixes, and names from the specifications.
- **Programs.** The guard around each `main`, its registration with the test runner, and a test for each module.
- **Protocol.** That behaviour comes from the specifications, the section each `@file` block cites, and the design notes.
- **Text.** That nothing from MISRA or the specifications is pasted into the project.
- **Working.** That the checks were run, that findings were fixed in the code, and that a changed check's own tests were run.